

Assignment-2

Part I

Q1. In SQL, NULL means a value is unknown or missing. When you compare something to NULL using =, the result is always UNKNOWN, not TRUE or FALSE. Since WHERE only keeps rows where the condition is TRUE, no rows pass, so COUNT(*) gives 0 even if many students actually have no phone number.

Corrected Query: `SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;`

Functions like COUNT(column), SUM, AVG, MIN, and MAX automatically ignore NULLs, because trying to do math with an unknown value wouldn't give a meaningful answer.

Q2. The student is right.

DISTINCT isn't needed here. With GROUP BY department, the query already gives one row per department. DISTINCT only removes duplicate rows, but there are no duplicates to remove since each department shows up just once. So DISTINCT does nothing useful — it's just extra clutter.

Q3. LEFT JOIN gives more rows than INNER JOIN, which tells us some customers in the Customer table have no matching rows in the Order table — meaning they've never placed an order. INNER JOIN drops these customers because it only keeps rows that match in both tables.

For example: say Customer has 500 rows, but only 380 distinct customers show up in Order. INNER JOIN gives 380 rows (assuming one order each). LEFT JOIN gives all 500 rows — 380 with order info filled in, and the other 120 with NULLs in the order columns, since those customers never ordered anything.

Q4. In standard SQL, every column you SELECT must either be in the GROUP BY clause or be inside an aggregate function. Here, employee_name is neither.

Once you group by department, each group has several employees in it. There's no single correct employee_name to show for that group. SQL would have to randomly pick one — and that pick wouldn't actually correspond to the AVG(salary) shown, so the result would be misleading. Some databases (like older MySQL with sql_mode="") allow this anyway, but the name shown is unpredictable.

Corrected Query: `SELECT department, AVG(salary) FROM Employee GROUP BY department;`

Q5. Error — AVG(salary) is an aggregate function, so it can't be used inside WHERE.

SQL runs in this order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

WHERE filters rows one at a time, before grouping happens — so at that point, groups don't exist yet and aggregate values haven't been calculated.

Corrected query: `SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING AVG(salary) > 80000;`

Q6.

- a. Products priced above the overall average → Non-correlated subquery. The overall average price is just one number, calculated once across the whole table — it doesn't change based on which row you're looking at. A non-correlated subquery calculates it once and reuses it, so it's faster.

`SELECT * FROM Product WHERE price > (SELECT AVG(price) FROM Product);`

- b. Employees earning more than their own department's average → Correlated

subquery.

The average is different for each department, so it depends on which employee row you're checking. The subquery has to run again for every row in the outer query, using that row's department — that's what makes it correlated.

```
SELECT e1.employee_name, e1.department, e1.salary FROM Employee e1 WHERE
e1.salary > (SELECT AVG(e2.salary) FROM Employee e2 WHERE e2.department =
e1.department);
```

A non-correlated subquery runs just once, and the same result gets reused for every outer row. A correlated subquery runs once *per row* of the outer query — so with N rows, it runs N times. On big tables, that's a lot of repeated work and can really slow things down. Because of this, correlated subqueries on large datasets are often rewritten as JOINS with aggregation instead.

Part II

(a) `SELECT c.customer_id, c.name, c.email, c.phone FROM Customer c LEFT JOIN "Order" o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;`

Why: LEFT JOIN keeps every customer; WHERE o.order_id IS NULL then picks out the ones with no matching order. An INNER JOIN would have quietly dropped them instead.

(b) `SELECT o.restaurant_id, r.name AS restaurant_name, AVG(order_total) AS avg_order_value FROM (SELECT oi.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS order_total FROM OrderItem oi JOIN "Order" o ON oi.order_id = o.order_id JOIN MenuItem mi ON oi.item_id = mi.item_id GROUP BY oi.order_id, o.restaurant_id) AS order_totals JOIN Restaurant r ON order_totals.restaurant_id = r.restaurant_id GROUP BY o.restaurant_id, r.name HAVING COUNT(*) > 5;`

Why: The inner subquery first works out the total for each order; then the outer query averages those totals per restaurant. HAVING is used (not WHERE) because the row-count filter happens after the grouping.

(c) `WITH order_totals AS (SELECT oi.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS order_total FROM OrderItem oi JOIN "Order" o ON oi.order_id = o.order_id JOIN MenuItem mi ON oi.item_id = mi.item_id GROUP BY oi.order_id, o.restaurant_id), restaurant_avg AS (SELECT restaurant_id, AVG(order_total) AS avg_order_value, COUNT(*) AS num_orders FROM order_totals GROUP BY restaurant_id HAVING COUNT(*) > 5) SELECT ra.restaurant_id, r.name, ra.avg_order_value FROM restaurant_avg ra JOIN Restaurant r ON ra.restaurant_id = r.restaurant_id WHERE ra.avg_order_value > (SELECT AVG(order_total) FROM order_totals);`

Why: The platform-wide average is one fixed number — a non-correlated subquery calculates it once and reuses it for each row. A correlated subquery would needlessly recalculate it for every restaurant.

(d) `SELECT mi.restaurant_id, oi.item_id, mi.item_name, SUM(oi.quantity) AS total_quantity FROM OrderItem oi JOIN MenuItem mi ON oi.item_id = mi.item_id GROUP BY mi.restaurant_id, oi.item_id, mi.item_name HAVING SUM(oi.quantity) = (SELECT MAX(sub.total_qty) FROM (SELECT oi2.item_id, SUM(oi2.quantity) AS total_qty FROM OrderItem oi2 JOIN MenuItem mi2 ON oi2.item_id = mi2.item_id WHERE mi2.restaurant_id = mi.restaurant_id GROUP BY oi2.item_id) sub);`

The correlated subquery finds the highest total quantity sold for items in that same restaurant; HAVING then keeps only the item(s) that match that highest value (so ties are all included correctly).

Limitation of GROUP BY + MAX for top-N per group: MAX() only gives you the highest value, not the whole row it belongs to — so you need an extra join to figure out which item actually has that value. If there's a tie, this approach can either return too many rows or miss some, depending on how the query is built. For a more reliable way to get the top-N per group (especially when N is more than 1), window functions like RANK() or ROW_NUMBER() with PARTITION BY are the better tool.

Part III

Q1. - student_id → student_name - (student_id, course_id) → grade - (student_id, course_id) → enrollment_date - course_id → course_name - course_id → instructor_id - instructor_id → instructor_name - instructor_id → instructor_office

Q2. Yes. Every column holds just one value, nothing repeats, and there's a primary key (student_id, course_id). So it satisfies 1NF.

Q3. There are several partial dependencies, since some columns only need part of the composite key:

- student_id → student_name (only needs student_id)
- course_id → course_name, instructor_id, instructor_name, instructor_office (only needs course_id)

Concrete anomaly example — Update anomaly: If a student's name changes, you'd have to update it in every row that student appears in. If you miss even one row, you end up with two different names for the same student_id.

Insertion anomaly: You can't add a new student until they sign up for a course, since course_id is part of the primary key and can't be left blank.

Deletion anomaly: If you remove a student's only enrollment, their name disappears from the database completely.

Q4. course_id → instructor_id → instructor_name, instructor_office

instructor_name and instructor_office depend on instructor_id, not directly on the primary key — that's a transitive dependency through a non-key column.

Anomaly: If an instructor's office changes, you'd need to update it in every course row taught by that instructor. And if a course gets deleted, that instructor's office info is lost for good.

Q5. Student(student_id (PK), student_name) Why: student_id → student_name

Course(course_id (PK), course_name, instructor_id (FK)) Why: course_id → course_name; course_id → instructor_id

Instructor(instructor_id (PK), instructor_name, instructor_office) Why: instructor_id → instructor_name; instructor_id → instructor_office

Enrollment(student_id (FK, PK), course_id (FK, PK), grade, enrollment_date) Why: (student_id, course_id) → grade; (student_id, course_id) → enrollment_date

Part IV

JOIN vs. Subquery Choice

To find customers who never placed an order, I could have used NOT IN with a subquery instead of LEFT JOIN. But NOT IN breaks if the subquery returns even a single NULL — it makes the whole outer query return nothing, which is a sneaky bug. LEFT JOIN with WHERE o.order_id IS NULL avoids that NULL problem and is just as easy to read, so it was the better pick.

Normalization Trade-offs: 3NF vs. Denormalization

After splitting the Enrollment table into Student, Course, Instructor, and Enrollment, a query like "list all students with their course name and instructor's office" now needs a four-table JOIN instead of a simple single-table SELECT. That adds complexity and can slow down reads on large datasets if there isn't good indexing.

Because of this, real-world systems often accept some denormalization. Data warehouses and reporting tools commonly use denormalized designs (like star schemas) to skip costly joins at query time. Even regular transactional systems sometimes keep a separate denormalized, read-friendly table alongside the normalized one — trading some duplicate data for faster, simpler queries.

